

Operations Research

Lecture 1: From Decisions to Models

A textbook treatment

Based on the lecture notes of El Mehdi Saad

October 1, 2026

Contents

How to read this book	2
1 From Decisions to Models	3
1.1 What Operations Research Is	3
1.2 Mathematical Programs	7
1.3 Linear Programs	9
1.4 Integer and Nonlinear Models	14
1.5 Bounds: How Good Is Our Solution?	16
1.6 Modelling with Graphs	19
1.7 Convexity	26
Notes on the original slides	29
Chapter takeaways	30

How to read this book

Each concept in this book is presented in the same rhythm.

- 1. Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
- 2. Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
- 3. Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
- 4. Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
- 5. Examples.** From “trivially small” to “genuinely useful.”
- 6. Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, “say it out loud” version of what is happening in the main text. Every major section closes with a summary box.

Prerequisites. Comfort with vectors, matrices, systems of linear equations, row reduction and the notions of rank and linear independence. Chapter 2 recalls exactly what is needed as it goes.

Chapter 1

From Decisions to Models

1.1 What Operations Research Is

Motivation

Every organisation makes decisions under limits: a train operator has a fixed fleet, a delivery company has a fixed number of vans, a hospital has a fixed number of nurses. The recurring question is

How should we use limited resources in the best possible way?

Operations Research (OR) answers this question with three ingredients: *mathematical models* that describe the situation, *data* that fill in the numbers, and *algorithms* that find good decisions. The word “research” in the name is historical: the field was born from research *on operations*, that is, from scientists studying how real systems actually behaved.

A little history: Blackett’s Circus

In 1940 German aircraft attacked Britain at night. Radar could locate an aircraft in the dark, but detection alone did not produce a hit. Between the radar and the gun sat a chain of steps: operators interpreted the echo, transmitted the information, a “predictor” estimated the future position, and only then was the gun aimed and fired. The aircraft moved, the information was imperfect, and every step took time. A small error or delay anywhere in the chain made the shot miss.

The birth of OR: do not study the radar alone, study the whole chain from detection to the fired shot.

In August 1940 the physicist Patrick Blackett joined Anti-Aircraft Command as scientific adviser and assembled a team of scientists from many disciplines, nicknamed *Blackett’s Circus*. Their method was the template for everything that followed:

observe the operation $\longrightarrow$ collect and analyse data $\longrightarrow$ improve how the system works.

Similar wartime teams studied convoy protection, aircraft patrols and military supply. After the war, these practical problems met new mathematics: John von Neumann’s game theory and, in 1947, George Dantzig’s *simplex method* for linear programming. From the 1950s onward OR spread to business, industry and public services: transport (which routes, which timetables?), healthcare (how many staff, how to cut waiting times?), energy (how much to produce and store?). In Morocco, for instance, ONCF must plan timetables and trains to serve passengers, and Amana must organise parcel routes to reduce travel while meeting deadlines.

The three ingredients of every model

Definition 1.1.1 (Ingredients of a decision model). A decision model consists of

- **decision variables:** the quantities we are free to choose;
- **constraints:** the conditions that limit our choices;
- **an objective:** a quantity we want to minimise or maximise.

Interpretation & discussion

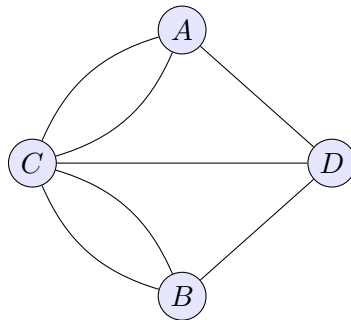
Every modelling exercise in this course, without exception, begins by answering three questions in plain language before writing a single formula: *What can we decide? What do we want to improve? What limits must we respect?* If you cannot answer these in words, you are not ready to write equations.

An earlier precursor: the Königsberg bridges

Long before wartime OR, in 18th-century Königsberg, seven bridges connected four land areas across the river Pregel. Citizens asked: *can you take a walk that crosses every bridge exactly once?* Start and finish anywhere you like.

Euler's idea (1736) was to keep only the connections: land areas become points, bridges become links. Four land areas become four points; seven bridges become seven links. The question becomes: *can we traverse every link exactly once?*

Euler's move: throw away everything except what is connected to what.



This abstraction is powerful enough that we can settle the question completely. The relevant notions (graph, degree, path) are introduced properly in Section 1.6; we state the theorem here because it is the historical origin of the whole “model it as a graph” philosophy, and its proof is short and beautiful. Feel free to read it now or after Section 1.6.

Theorem 1.1.2 (Euler, 1736). *Let G be a finite connected graph (multiple edges between two vertices are allowed). A walk that uses every edge exactly once exists if and only if the number of vertices with odd degree is 0 or 2. If it is 0, every such walk starts and ends at the same vertex; if it is 2, every such walk starts at one odd vertex and ends at the other.*

Proof. Step 1 (necessity). Suppose such a walk W exists. Each time W passes through a vertex v , it arrives on one edge and leaves on another, consuming two edges at v . If v is neither the first nor the last vertex of W , all edges at v are consumed in such pairs, so $\deg v$ is even. If v is the first vertex but not the last, one extra edge (the first) is consumed alone, so $\deg v$ is odd; similarly for the last vertex. If the first and last vertices coincide, the first and last edges pair up and

that vertex also has even degree. Hence the odd-degree vertices are exactly the endpoints when they differ, giving 0 or 2 odd vertices.

Step 2 (sufficiency, all degrees even). Assume every degree is even. Start at any vertex v_0 and walk along unused edges as long as possible, never reusing an edge. Whenever we *enter* a vertex $v \neq v_0$ we have used an odd number of its edges, so an unused one remains and we can leave. Therefore the walk can only get stuck at v_0 : it is a closed trail T_1 . If T_1 uses all edges we are done. Otherwise, because G is connected, some vertex u on T_1 has an unused edge. Removing the edges of T_1 leaves every degree even, so the same argument from u produces a closed trail T_2 of unused edges starting and ending at u . Splice T_2 into T_1 at u to obtain a longer closed trail. Repeat; since the edge set is finite the process ends with a closed trail using all edges.

Step 3 (sufficiency, exactly two odd vertices p, q). Add a new edge $\{p, q\}$. Now every degree is even, so by Step 2 there is a closed trail using all edges. Delete the added edge from that trail: what remains is a walk from p to q using every original edge exactly once. $\square$

Key insight

Count *pairs*. Passing through a vertex uses two of its edges, so parity of the degree is the only thing that matters. Euler's theorem is the first example of a recurring OR theme: a necessary condition (parity) that turns out to be sufficient as well, giving a complete characterisation.

Example 1.1.3 (Königsberg has no such walk). In the graph above, the degrees are $\deg A = 3$, $\deg B = 3$, $\deg C = 5$, $\deg D = 3$. Four vertices have odd degree, so by Theorem 1.1.2 no walk crosses every bridge exactly once. Two centuries of citizens failed not for lack of cleverness but because the task was impossible, and the proof took one line once the right model was chosen.

Modelling: translating a real decision

Definition 1.1.4 (Mathematical modelling). *Mathematical modelling* means translating a real-world problem into mathematical terms while keeping the features that matter for the decision. The translation table is

Real-world question	Mathematical ingredient
What can we decide?	Decision variables
What do we want to improve?	Objective function
What limits must we respect?	Constraints

Optimisation then finds the best decision allowed by the model. Afterwards we interpret the result and check whether it makes sense in reality. Two points deserve emphasis.

- **The model is a choice.** The same real problem can be modelled in several ways, and the choice affects both how hard the model is to solve and how useful the answer is.
- **Start simple.** Keep the essential decisions, constraints and objective. A simple model is easier to solve and to understand; extra detail (truck capacities, uncertain demand, ...) is worth adding *only if it changes the*

decision. Choose the criterion carefully too: the shortest route need not be the fastest. When several objectives compete, set priorities, use weights, or compare trade-offs.

Example 1.1.5 (Fertiliser delivery, OCP-inspired). Two plants (A and B) must supply two customers (1 and 2) with the same fertiliser this week. Either plant can serve either customer. Plant A has 700 t available, plant B has 500 t; customer 1 needs 600 t, customer 2 needs 400 t. Transport costs in MAD per tonne are

MAD/t	Customer 1	Customer 2	Available
Plant A	100	180	700 t
Plant B	140	80	500 t
Demand	600 t	400 t	

What can we decide? How many tonnes to send on each route: $x_{A1}, x_{A2}, x_{B1}, x_{B2}$. *What do we want to improve?* Total transport cost. *What limits?* Meet every order without exceeding any plant's stock. The model:

$$\begin{aligned}
 \min \quad & 100x_{A1} + 180x_{A2} + 140x_{B1} + 80x_{B2} \\
 \text{s.t.} \quad & x_{A1} + x_{A2} \leq 700, \quad x_{B1} + x_{B2} \leq 500 \quad (\text{available supply}) \\
 & x_{A1} + x_{B1} = 600, \quad x_{A2} + x_{B2} = 400 \quad (\text{meet demand}) \\
 & x_{A1}, x_{A2}, x_{B1}, x_{B2} \geq 0 \quad (\text{no negative shipments}).
 \end{aligned}$$

We return to this model in Section 1.5, where we will prove which plan is best.

From a model to a working solution

The full OR workflow has four steps: (1) **model** (decisions, constraints, objective, assumptions); (2) **choose an algorithm**, balancing solution quality, speed and implementation effort; (3) **validate**, through theoretical guarantees and tests on realistic data; (4) **deploy**, building reliable software that is monitored and maintained. If validation fails, revisit the model or the algorithm. This course concentrates on steps 1 and 2: modelling, then solution methods, starting with linear programming and continuing with graph algorithms.

Summary: What Operations Research is

- OR uses models, data and algorithms to make better decisions under limited resources.
- Every model has decision variables, constraints and an objective. Say them in words first.
- Modelling is a choice: start simple, add detail only when it changes the decision, choose the criterion with care.
- Euler's bridges problem is the prototype: strip the situation to its structure, and an impossible-looking question becomes a one-line parity argument.

1.2 Mathematical Programs

Motivation

Once decisions, constraints and objective are identified, every model in this course has the same abstract shape: *choose the best element of an allowed set*. Giving this shape a name lets us talk about feasibility, optimality and the different things that can go wrong before we specialise to any particular type of model.

Definition 1.2.1 (Mathematical program). A *mathematical program* is a problem of the form

$$\min_{x \in X} f(x),$$

where

- x is the *decision*, a vector of quantities we can choose;
- X is the *feasible set*, the set of all choices allowed by the constraints; an $x \in X$ is called *feasible*;
- $f(x)$ is the *objective value* of the choice x .

An *optimal solution* is a feasible $x^* \in X$ with $f(x^*) \leq f(x)$ for every $x \in X$; the number $f(x^*)$ is the *optimal value*. Maximisation problems $\max_{x \in X} p(x)$ are defined in the same way with the inequality reversed.

A maximisation problem is never a genuinely different object, as the next proposition shows.

Proposition 1.2.2 (Max is min of the negative). *For any set X and function $p : X \rightarrow \mathbb{R}$, the point x^* maximises p over X if and only if it minimises $-p$ over X , and $\max_X p = -\min_X (-p)$ whenever either side exists.*

Maximising profit is minimising loss. Flipping the sign lets every theorem be stated once.

Proof. For all $x \in X$: $p(x^*) \geq p(x) \iff -p(x^*) \leq -p(x)$. So x^* is a maximiser of p exactly when it is a minimiser of $-p$. Evaluating gives $\max_X p = p(x^*) = -(-p(x^*)) = -\min_X (-p)$. $\square$

The domain is part of the model

Definition 1.2.3 (Variable types). A decision variable is

- *continuous* if it may take any real value in an interval, e.g. tonnes shipped, $x \geq 0$;
- *integer* if it must lie in $\{0, 1, 2, \dots\}$ (or $\mathbb{Z}$), e.g. number of trucks;
- *binary* if it must lie in $\{0, 1\}$, e.g. “open a warehouse?”.

Interpretation & discussion

Is 2.4 tonnes meaningful? Yes. Is 2.4 trucks meaningful? No. The domain of a variable is not a technicality: it encodes a physical fact, and getting it wrong produces answers that cannot be implemented. A useful check on any finished model: *does every mathematically allowed choice make sense in reality?*

Three possible outcomes

Definition 1.2.4 (Outcomes of a mathematical program). A minimisation problem $\min_{x \in X} f(x)$ is

- *infeasible* if $X = \emptyset$;
- *unbounded* if $X \neq \emptyset$ and for every $M \in \mathbb{R}$ there is $x \in X$ with $f(x) < M$ (the objective can be made arbitrarily good);
- *solvable* if an optimal solution exists.

For a maximisation problem, “unbounded” means f can be made arbitrarily large.

Example 1.2.5.

- *Infeasible*: requiring $x \geq 5$ and $x \leq 3$ allows no choice at all.
- *Unbounded*: maximising x subject only to $x \geq 0$ has no finite best value.
- *Solvable*: the production model of Section 1.3 has best profit 100 MAD.

Remark 1.2.6 (The three outcomes are not exhaustive). A problem can be feasible, bounded, and still have no optimal solution: $\min x$ over $X = \{x \in \mathbb{R} : x > 0\}$ has infimum 0 but no feasible point achieves it. For linear programs this pathology never happens (Chapter 2 proves it), which is one reason linear programs are so pleasant.

Interpretation & discussion

An unexpected outcome is a *message about the model*, not a failure of the mathematics. Infeasibility usually means inconsistent data or a forgotten degree of freedom; unboundedness almost always means a missing limit (nobody can really sell infinitely many litres).

Recognising the structure

Definition 1.2.7 (Classes of mathematical programs).

- A *linear program* (LP) has continuous variables and an objective and constraints that are all *linear* expressions (weighted sums of the variables with constant coefficients).
- An *integer program* (IP) or *mixed-integer program* (MIP) has linear expressions, with some or all variables restricted to integers.
- A *nonlinear program* (NLP) has at least one nonlinear term such as x^2 or xy in the objective or the constraints.

The structure decides the solution method: linear programs have fast, exact algorithms (Chapter 2 begins this story); integer programs are much harder in general; nonlinear programs range from easy (convex, see Section 1.7) to hopeless.

Technique: Spotting the class in five seconds

- Any product of two variables, any square, any $\sqrt{\cdot}$, $\log$, $|\cdot|$ of a variable: **nonlinear**.

- A coefficient that itself depends on a variable (“price drops as quantity rises”): **nonlinear**, even if it looks like a product of constant and variable.
- Everything linear but some variable in $\{0, 1\}$ or $\mathbb{Z}$: **integer/mixed-integer**.
- Everything linear, all variables continuous: **LP**.

The liquid-production model below has linear expressions and continuous litres, so it is an LP.

Summary: Mathematical programs

- $\min_{x \in X} f(x)$: decision x , feasible set X , objective f . Optimal means feasible and best.
- Maximising p is minimising $-p$ (Proposition 1.2.2); the optimal value flips sign.
- The domain (continuous, integer, binary) is part of the model.
- Outcomes: an optimum, infeasible, or unbounded. Interpret unexpected outcomes as messages about the model.
- Class (LP, IP/MIP, NLP) is decided by linearity and by variable domains, and it decides the algorithm.

1.3 Linear Programs

Motivation

Linear programs are the workhorse of OR: they are expressive enough to model production, transport, blending, scheduling and flows, and they can be solved exactly and quickly even with millions of variables. To talk about “the” LP algorithm we need a single agreed shape for every LP. This section fixes a standard form and proves that *every* LP can be rewritten in it without changing the decisions or their costs.

Definition 1.3.1 (Linear program, standard (canonical) form). A *linear program in standard form* (this course’s convention) is

$$\begin{aligned} &\text{maximise} && c_1x_1 + \cdots + c_nx_n \\ &\text{subject to} && a_{i1}x_1 + \cdots + a_{in}x_n \leq b_i \quad (i = 1, \dots, m), \\ &&& x_1, \dots, x_n \geq 0. \end{aligned}$$

The numbers a_{ij}, b_i, c_j are given data; only the x_j are chosen. The convention is: *maximise*, all constraints of type $\leq$, all variables nonnegative (zero allowed).

Remark 1.3.2 (Conventions are not definitions). The directions “max”, “ $\leq$ ”, “ ≥ 0 ” are a convention, not the definition of linearity. An LP is any problem with a linear objective and linear equality or inequality constraints on continuous variables. Other textbooks call $Ax = b, x \geq 0$ the standard form (Chapter 2 calls it the *equational form*). Both conventions represent exactly the same problems.

Real decisions do not always fit the format

Example 1.3.3 (Electricity for a workshop). A workshop needs at least 3 kWh during the next hour. It can generate locally, $g \geq 0$ kWh at 5 MAD/kWh, and it exchanges with the grid a net amount t with $-2 \leq t \leq 2$: $t > 0$ means buying at 2 MAD/kWh, $t < 0$ means selling and earning 2 MAD/kWh. The model is

$$\min 5g + 2t \quad \text{s.t.} \quad g + t \geq 3, \quad -2 \leq t \leq 2, \quad g \geq 0.$$

This is an LP: every expression is linear and the variables are continuous. But it minimises, has a $\geq$ constraint, and has a variable t that may be negative.

Three rewrites

Three elementary rewrites bring any LP into standard form. Each is justified by a small proposition stating that the feasible choices and their objective values correspond exactly.

Nothing is “solved” during a rewrite. We only translate; the decisions and their costs are the same before and after.

Proposition 1.3.4 (Reverse a sign). *For any real numbers, $a^\top x \geq b \iff -a^\top x \leq -b$. Hence $a \geq$ constraint can be replaced by $a \leq$ constraint with the same solution set.*

Proof. Multiplying an inequality by -1 reverses it: $u \geq v \iff -u \leq -v$. Apply with $u = a^\top x$, $v = b$. $\square$

Proposition 1.3.5 (Split a free variable). *Let $t \in \mathbb{R}$ be unrestricted in sign. The map $(u, v) \mapsto u - v$ from $\{(u, v) : u \geq 0, v \geq 0\}$ onto $\mathbb{R}$ is surjective. Consequently, replacing every occurrence of t by $u - v$ and adding $u, v \geq 0$ yields a problem whose feasible $(u, v, \dots)$ correspond to feasible $(t, \dots)$ via $t = u - v$, with equal objective values; the optimal values of the two problems coincide.*

Proof. Surjectivity. Given $t \in \mathbb{R}$, take $(u, v) = (t, 0)$ if $t \geq 0$ and $(u, v) = (0, -t)$ if $t < 0$; both are nonnegative and $u - v = t$.

Correspondence. Every constraint and the objective are functions of t (and the other variables). Substituting $t = u - v$ is an identity, so a point $(u, v, \dots)$ with $u, v \geq 0$ satisfies the new constraints exactly when $(u - v, \dots)$ satisfies the old ones, and the objective values agree. Every old feasible point is reached by surjectivity; every new feasible point maps to an old one. The two problems therefore have the same set of attainable objective values, hence the same optimal value. $\square$

Gotcha

The representation $t = u - v$ is *not unique*: $t = 2$ is $(2, 0)$ but also $(5, 3)$. This is harmless for the optimal value, but do not be surprised when a solver returns both $u > 0$ and $v > 0$; the decision is still $t = u - v$. Also, keep the original bounds on t : for $-2 \leq t \leq 2$ we must add $u - v \leq 2$ and $-u + v \leq 2$.

Proposition 1.3.6 (Min to max). *$\min f(x)$ over X and $\max(-f(x))$ over X have the same optimal solutions, and the minimum equals the negative of the maximum.*

Proof. This is Proposition 1.2.2 applied to $p = -f$. $\square$

Example 1.3.7 (The electricity problem in standard form). Start from Example 1.3.3. Reverse the sign of the supply constraint: $g + t \geq 3 \iff -g - t \leq -3$. Split $t = u - v$ with $u, v \geq 0$, keeping the bounds $u - v \leq 2$ and $-u + v \leq 2$. Flip the objective: $\min(5g + 2t) \iff \max(-5g - 2t)$. Substituting everywhere:

$$\begin{array}{llll} \text{maximise} & -5g - 2u + 2v & & \\ \text{subject to} & -g - u + v \leq -3 & & \text{(minimum supply)} \\ & u - v \leq 2 & & \text{(import limit)} \\ & -u + v \leq 2 & & \text{(export limit)} \\ & g, u, v \geq 0. & & \end{array}$$

Afterwards we recover the real decision as $t = u - v$, and the original minimum cost is the negative of this maximum. Same choices, same costs, after translation; no solving was needed.

For the record, the optimum is easy to see: buying from the grid (2 MAD) is cheaper than generating (5 MAD), so buy the maximum $t = 2$ and generate $g = 1$, for a cost of 9 MAD. Section 1.5 shows how to *prove* that nothing beats 9.

A small production problem, from words to a picture

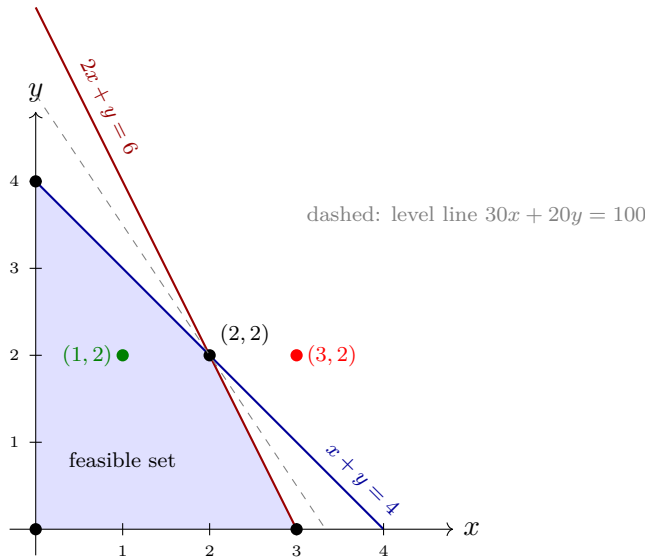
Example 1.3.8 (Liquid products). A workshop makes two liquid products, A and B . Per litre, A needs 1 hour of machine time and 2 kg of raw material and earns 30 MAD; B needs 1 hour and 1 kg and earns 20 MAD. Today 4 hours and 6 kg are available. Assumptions: production is divisible, and everything produced can be sold.

Decisions: x = litres of A , y = litres of B . *Objective:* $\max 30x + 20y$ (MAD).

Constraints:

$$x + y \leq 4 \text{ (machine hours)}, \quad 2x + y \leq 6 \text{ (raw material, kg)}, \quad x, y \geq 0.$$

Check the units: both sides of each constraint are hours, or kilograms. Check the direction: “at most” gives $\leq$.



Feasible means respecting every constraint. The plan (1, 2) uses 3 hours and 4 kg: feasible. The plan (3, 2) would use 5 hours and 8 kg: infeasible, however profitable it looks. A profitable plan is useless if it violates the constraints.

Optimal means best among feasible plans. The dashed line $30x + 20y = 100$ is a *level line*: all plans on it earn the same profit. Sliding it upward increases profit; the last feasible point it touches is the corner (2, 2), with profit $30(2) + 20(2) = 100$ MAD. The picture is convincing, but a picture is not a proof. Here is the proof, and it contains one of the central ideas of linear programming.

Proposition 1.3.9 (A certificate of optimality for Example 1.3.8). *Every feasible plan satisfies $30x + 20y \leq 100$. Hence (2, 2) is optimal.*

Proof. For any feasible (x, y) ,

$$30x + 20y = 10(x + y) + 10(2x + y) \leq 10 \cdot 4 + 10 \cdot 6 = 100,$$

using $x + y \leq 4$ and $2x + y \leq 6$ and the fact that we multiplied both by *nonnegative* numbers, which preserves the inequalities. Since (2, 2) is feasible and achieves 100, it is optimal. $\square$

Key insight

To prove no plan beats 100, we did not examine plans at all. We *combined the constraints* with nonnegative weights (10 and 10) so that the combination reproduces the objective exactly. The weights are a *certificate*: anyone can check the two-line computation. This idea, made general, is the next lemma; made fully general it becomes LP duality later in the course.

Lemma 1.3.10 (Weak duality, elementary form). *Consider the standard-form LP $\max\{c^\top x : Ax \leq b, x \geq 0\}$ with $A \in \mathbb{R}^{m \times n}$. If $y \in \mathbb{R}^m$ satisfies $y \geq 0$ and $A^\top y \geq c$ (componentwise), then $c^\top x \leq b^\top y$ for every feasible x . In particular, if some feasible x^* has $c^\top x^* = b^\top y$, then x^* is optimal.*

Proof. Let x be feasible. Since $x \geq 0$ and $A^\top y \geq c$, multiplying componentwise and summing gives $c^\top x \leq (A^\top y)^\top x = y^\top (Ax)$. Since $y \geq 0$ and $Ax \leq b$, the same argument gives $y^\top (Ax) \leq y^\top b$. Chain the two: $c^\top x \leq b^\top y$. If $c^\top x^* = b^\top y$

for a feasible x^* , then $c^\top x \leq c^\top x^*$ for all feasible x , which is the definition of optimality. $\square$

Interpretation & discussion

In Example 1.3.8, $A = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}$, $b = (4, 6)^\top$, $c = (30, 20)^\top$ and $y = (10, 10)^\top$ gives $A^\top y = (30, 20)^\top = c$ and $b^\top y = 100$. The two places where signs matter are exactly the two places where the proof used nonnegativity: we need $x \geq 0$ to pass from $c \leq A^\top y$ to $c^\top x \leq y^\top Ax$, and $y \geq 0$ to pass from $Ax \leq b$ to $y^\top Ax \leq y^\top b$. Forgetting either is the classic error.

Technique: Solving a two-variable LP by hand

1. Draw each constraint as a line; shade the half-plane it allows. The feasible set is the intersection (a polygon, possibly unbounded or empty).
2. Draw one level line of the objective, $c_1x + c_2y = k$; the direction of increase is the vector (c_1, c_2) .
3. Slide the level line in the direction of increase until it is about to leave the feasible set. The last contact is at a corner (or along a whole edge if the level line is parallel to it).
4. Compute the objective at the candidate corners and pick the best. *Shortcut:* corners are intersections of two constraint lines, so solve 2×2 systems.
5. To *prove* optimality on an exam, exhibit nonnegative multipliers of the tight constraints that reproduce the objective (Lemma 1.3.10).

Classic problems

- *Corners of Example 1.3.8.* $(0, 0), (3, 0), (2, 2), (0, 4)$ with profits 0, 90, 100, 80. Best: $(2, 2)$.
- Write $\min 2x - y$ s.t. $x - y \geq 1$, $x \in \mathbb{R}$, $y \geq 0$ in standard form. $\max -2u + 2v + y$ s.t. $-u + v + y \leq -1$, $u, v, y \geq 0$, with $x = u - v$.
- Is $\{x : |x| \leq 1\}$ an LP constraint? Yes: $|x| \leq 1 \iff x \leq 1$ and $-x \leq 1$. But $|x| \geq 1$ is *not* (the set is two disjoint pieces; see convexity, Section 1.7).
- *Certificate for the electricity problem.* From $g + t \geq 3$ (weight 5) and $t \leq 2$ (weight 3): $5g + 5t \geq 15$ and $-3t \geq -6$; adding, $5g + 2t \geq 9$. The plan $g = 1, t = 2$ costs 9, so it is optimal.

Gotcha

- Reading “at least” as $\leq$. Always re-read each constraint in words after writing it.
- Forgetting $x, y \geq 0$. Without them the production LP is unbounded (produce -1000 litres of B !).
- Treating a level line as a constraint. Level lines are where the objective

is constant; they do not restrict anything.

- Concluding optimality from the picture alone. On an exam, give the certificate or evaluate all corners with justification.

Summary: Linear programs

- Standard form: maximise a linear objective, all constraints $\leq$, all variables ≥ 0 .
- Any LP reaches standard form by three rewrites: reverse signs of $\geq$ constraints, split free variables as $u - v$, and turn min into max of the negative. Each is an exact translation (Propositions 1.3.4–1.3.6).
- Feasible = respects every constraint; optimal = best among feasible.
- Weak duality (Lemma 1.3.10): a nonnegative combination of constraints that dominates the objective is a checkable proof of optimality.

1.4 Integer and Nonlinear Models

Motivation

Not every decision is a divisible quantity. Some are yes/no choices (take an item, fund a project, open a warehouse); some interact multiplicatively (area is length times width). These produce integer and nonlinear programs. They are usually harder to solve than LPs, but writing them correctly is the same discipline: decisions, objective, constraints, domain.

Integer programming: pack a backpack

Example 1.4.1 (Knapsack). Your essentials are packed and 5 kg of capacity remain for optional gear. Each item is available once: take it whole or leave it.

Item	Weight (kg)	Usefulness
Camera kit	3	9
Binoculars	2	6
Warm blanket	4	7
Book	1	2

Define $z_i = 1$ if item i is taken and $z_i = 0$ otherwise, in the order camera, binoculars, blanket, book. The model is

$$\max 9z_1 + 6z_2 + 7z_3 + 2z_4 \quad \text{s.t.} \quad 3z_1 + 2z_2 + 4z_3 + z_4 \leq 5, \quad z_i \in \{0, 1\}.$$

All expressions are linear; the decisions are binary integers. This is the *knapsack problem*, a binary integer program.

Proposition 1.4.2 (Solution of Example 1.4.1). *The optimal selection is camera and binoculars, with usefulness 15.*

Proof. There are $2^4 = 16$ selections; we may organise the check. The blanket (4

kg) can be combined only with the book: usefulness 9, or alone: 7. Without the blanket, the items weigh 3, 2, 1: all three weigh $6 > 5$, so at most two. Pairs: camera+binoculars (5 kg, 15), camera+book (4 kg, 11), binoculars+book (3 kg, 8). Singles give at most 9. The maximum over all feasible selections is 15. $\square$

Remark 1.4.3 (Why the domain matters). Replacing $z_i \in \{0, 1\}$ by $0 \leq z_i \leq 1$ gives an LP, the *linear relaxation*, whose solutions may take a fraction of an item. Here the relaxation happens to have the integer optimum 15 as well (fill by best usefulness-per-kilogram: camera 3, binoculars 3, then the bag is full), but in general the relaxation can be strictly better than any integer plan, and its fractional answer is not implementable. Section 1.5 explains how such relaxations are nevertheless useful as *bounds*.

Example 1.4.4 (Your turn: fund projects). A team can spend at most 7 budget units; each of projects A, B, C is funded in full or not at all.

	A	B	C
Cost	4	3	2
Benefit	6	5	4

Model: $a, b, c \in \{0, 1\}$ ($1 = \text{funded}$), $\max 6a + 5b + 4c$ s.t. $4a + 3b + 2c \leq 7$.

Solution: all three cost $9 > 7$. Pairs: $A+B$ costs 7, benefit 11; $A+C$ costs 6, benefit 10; $B+C$ costs 5, benefit 9. Singles give at most 6. Fund A and B : $(a, b, c) = (1, 1, 0)$, benefit 11.

Nonlinear programming: design a garden

Example 1.4.5 (Garden). A school has 40 m of fencing for a rectangular garden; all four sides must be fenced. Choose length $x \geq 0$ and width $y \geq 0$ (fractional metres allowed) to enclose the largest area using at most 40 m of fence:

$$\max xy \quad \text{s.t.} \quad 2x + 2y \leq 40, \quad x, y \geq 0.$$

The constraint $2x + 2y$ is linear but the objective xy multiplies two decision variables: the model is an NLP because its objective is nonlinear.

Proposition 1.4.6 (Solution of Example 1.4.5). *The maximum area is 100 m^2 , attained only by the square $x = y = 10$.*

Proof. For $x, y \geq 0$, $0 \leq (x-y)^2 = x^2 - 2xy + y^2$, so $4xy \leq x^2 + 2xy + y^2 = (x+y)^2$, i.e. $xy \leq \left(\frac{x+y}{2}\right)^2$, with equality iff $x = y$. The constraint gives $x + y \leq 20$, so $xy \leq 10^2 = 100$. The point $(10, 10)$ is feasible ($2 \cdot 10 + 2 \cdot 10 = 40$) and achieves 100. Equality in the chain forces $x = y$ and $x + y = 20$, so the square is the unique maximiser. $\square$

Key insight

The proof is again a *bound plus a plan that attains it*: the inequality $xy \leq ((x+y)/2)^2$ (the arithmetic-geometric mean inequality) is a lower-effort certificate than any calculus. The pattern “bound, then attain” is the subject of the next section.

Technique: Writing integer and nonlinear models

- A yes/no decision is a binary variable $z \in \{0, 1\}$. “Take the whole item or nothing” means *do not* relax to $[0, 1]$.
- Count things with integers, measure things with reals. Ask: would 2.4 of this make sense?
- Products of *data* and variables are linear ($3z_1$); products of two *variables* are not (xy).
- For tiny integer programs, enumerate cleverly: fix the heaviest item in or out, then treat the rest.
- For a symmetric NLP with a sum constraint, try AM–GM before calculus.

Gotcha

Rounding an LP solution is not a way to solve an integer program: the rounded point may be infeasible, and even when feasible it may be far from optimal. Keep the domain $\{0, 1\}$ or $\mathbb{Z}$ in the model and solve the integer problem as such.

Summary: Integer and nonlinear models

- Binary variables model select/assign/activate decisions; linear expressions with integer variables give an IP or MIP (knapsack, project funding).
- A single product of variables makes a model nonlinear (garden design).
- Small instances are solved by structured enumeration or by an inequality that bounds the objective; large instances need the algorithms of later chapters.

1.5 Bounds: How Good Is Our Solution?

Motivation

Finding the best solution can take a long time. Suppose we already have a feasible plan. Should we keep searching? The honest answer requires two numbers. For a cost-minimisation problem:

- **What cost can we actually achieve?** Any feasible plan gives an *upper bound* U on the optimal cost.
- **What cost can we prove is impossible to beat?** A *lower bound* L .

Then

$$L \leq \text{best possible cost} \leq U,$$

and the *gap* $U - L$ tells us how much improvement might remain. If $L = U$, we have proved optimality without examining every plan.

Two questions: what can we achieve? What can we prove impossible? The answers bracket the optimum.

Upper bounds from feasible plans

Proposition 1.5.1 (Feasible plans give upper bounds). *Let $z^* = \min_{x \in X} f(x)$ be the optimal value of a minimisation problem (assumed to exist). For every feasible $\bar{x} \in X$, $z^* \leq f(\bar{x})$.*

Proof. z^* is the minimum of f over X and $\bar{x} \in X$, so $z^* \leq f(\bar{x})$ by definition of a minimum. $\square$

Lower bounds from relaxations

Definition 1.5.2 (Relaxation). Let (P) be $\min_{x \in X} f(x)$. A problem (R) : $\min_{x \in X'} f'(x)$ is a *relaxation* of (P) if

$$X \subseteq X' \quad \text{and} \quad f'(x) \leq f(x) \quad \text{for all } x \in X.$$

That is, (R) allows at least the same choices and never charges more for them. The most common relaxations *drop constraints* ($X' \supseteq X$, $f' = f$) or *drop integrality* ($z \in \{0, 1\}$ becomes $0 \leq z \leq 1$).

Theorem 1.5.3 (Relaxations give lower bounds). *If (R) is a relaxation of (P) , then $\inf_{x \in X'} f'(x) \leq \inf_{x \in X} f(x)$. In words: the optimal value of the relaxation is a lower bound on the optimal value of the original problem. For maximisation problems, a relaxation gives an upper bound.*

Proof. Let $x \in X$ be any feasible point of (P) . Then $x \in X'$, so $\inf_{x \in X'} f' \leq f'(x) \leq f(x)$. The number $\inf_{x \in X'} f'$ is therefore a lower bound for the set $\{f(x) : x \in X\}$, and hence is at most its greatest lower bound $\inf_X f$. (If X is empty the right side is $+\infty$ and the inequality is trivial.) For maximisation, apply the same argument with $\geq$ throughout. $\square$

Key insight

Removing a constraint can only make the minimum smaller or equal: you are minimising over a bigger set. Making the objective cheaper can only make the minimum smaller or equal. Relaxations exploit exactly these two monotonicities.

Matching bounds prove optimality

Theorem 1.5.4 (Optimality by matching bounds). *Let (P) be $\min_{x \in X} f(x)$. Suppose $L \leq f(x)$ for every $x \in X$ (a valid lower bound) and $\bar{x} \in X$ satisfies $f(\bar{x}) = L$. Then $\bar{x}$ is optimal and the optimal value is L .*

Proof. For every $x \in X$, $f(\bar{x}) = L \leq f(x)$. This is precisely the definition of $\bar{x}$ being an optimal solution, and its value is L . $\square$

Interpretation & discussion

The gap $U - L$ is a guarantee, not a promise: our plan is at most $U - L$ worse than optimal, but it does *not* follow that all of $U - L$ can be saved. Either the plan may be better than we can prove (the lower bound is weak), or the lower bound may be tight and the plan poor. Closing the gap can happen

from either side: find a better plan (lower U) or a stronger argument (raise L).

Worked example: the fertiliser delivery problem

Example 1.5.5 (Upper bound: build a plan we can actually use). Return to Example 1.1.5. Propose: $A \rightarrow 1$: 500 t, $A \rightarrow 2$: 0, $B \rightarrow 1$: 100 t, $B \rightarrow 2$: 400 t. *Feasibility*: customer 1 receives $500 + 100 = 600$, customer 2 receives 400; plant A ships $500 \leq 700$, plant B ships $100 + 400 = 500 \leq 500$. *Cost*:

$$U = 500(100) + 0(180) + 100(140) + 400(80) = 96,000 \text{ MAD.}$$

By Proposition 1.5.1, the optimal cost is at most 96,000.

Example 1.5.6 (Lower bound: make the problem easier). Temporarily ignore the plant capacities but keep the demands. Each customer may now use its cheapest route, since nothing limits the plant. Customer 1 pays at least 100 MAD/t for 600 t, customer 2 at least 80 MAD/t for 400 t:

$$L = 600(100) + 400(80) = 92,000 \text{ MAD.}$$

Justify the direction. Dropping the two supply constraints enlarges the feasible set with the same objective, so this is a relaxation, and by Theorem 1.5.3 its optimal value 92,000 is a lower bound. (To be precise, we also need that 92,000 is the optimum of the relaxed problem: for any feasible x of the relaxation, $100x_{A1} + 140x_{B1} \geq 100(x_{A1} + x_{B1}) = 60,000$ and $180x_{A2} + 80x_{B2} \geq 80(x_{A2} + x_{B2}) = 32,000$, using nonnegativity of the variables.)

So $92,000 \leq \text{optimal cost} \leq 96,000$: the gap is 4,000 MAD. Our plan is at most 4,000 MAD more expensive than optimal.

Example 1.5.7 (Matching bounds). Can we achieve 92,000? Send 600 t from A to customer 1 and 400 t from B to customer 2, nothing on the other routes. Check the *original* constraints: both demands are met, and $600 \leq 700$, $400 \leq 500$. Cost = $600(100) + 400(80) = 92,000$. Now $U = 92,000 = L$, and by Theorem 1.5.4 this plan is optimal: no plan can beat it.

Key insight

We never enumerated plans, and we never ran an algorithm. A relaxation gave a number no plan can beat; a single plan reached it. This “sandwich” is how every modern optimisation solver reports progress, and it is how you should justify optimality on an exam whenever you can.

Technique: Producing bounds quickly

- *Upper bound (minimisation)*: any feasible plan. A greedy or “obvious” plan is fine; just *verify feasibility* explicitly.
- *Lower bound (minimisation)*: drop the constraints that make the problem hard, solve what is left. Typical drops: capacities, integrality, coupling constraints between subproblems.

- *Per-unit argument*: “each unit of demand costs at least $c_{\min}$ ” is a relaxation in disguise (it ignores everything except demand).
- *Combine constraints*: nonnegative combinations of constraints that dominate the objective give bounds directly (Lemma 1.3.10); for maximisation this yields upper bounds.
- Always *state the direction*: relaxations bound minima from below and maxima from above.

Classic problems

- *Knapsack bound*. In Example 1.4.1 the LP relaxation gives usefulness ≤ 15 ; the integer plan camera+binoculars achieves 15. Optimal by matching bounds.
- *Transport with a single cheapest route*. If every customer’s cheapest plant has enough capacity to serve it alone, then “everyone uses the cheapest route” is feasible and equals the relaxation bound, hence optimal. (That is exactly what happened above.)
- *Gap interpretation*. “ $L = 90$, $U = 96$: is the plan within 10% of optimal?” Yes. We know $U - z^* \leq U - L = 6$ and $z^* \geq L = 90$, so the relative excess is $(U - z^*)/z^* \leq 6/90 \approx 6.7\% < 10\%$.

Gotcha

- A relaxed solution is usually *infeasible* for the original problem. Its *value* is a bound; its *plan* may be nonsense (fractional items, exceeded capacities).
- Adding constraints does not give a lower bound for a minimisation; it gives an upper bound (you restrict the choices).
- For maximisation problems every direction flips: feasible plans give lower bounds, relaxations give upper bounds.

Summary: Bounds

- Minimisation: feasible plan $\Rightarrow$ upper bound; relaxation (drop constraints or cheapen the objective) $\Rightarrow$ lower bound (Theorem 1.5.3).
- $L \leq z^* \leq U$; the gap $U - L$ bounds the possible improvement.
- $L = U$ proves optimality (Theorem 1.5.4) with no enumeration.

1.6 Modelling with Graphs

Motivation

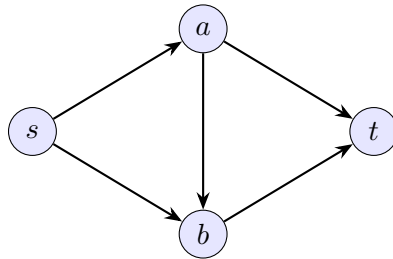
Many decisions depend on which objects are connected to which: cities and roads, tasks and precedence relations, people and the jobs they can do.

Setting	Objects	Connections
Transport	Cities	Roads
Project planning	Tasks	Precedence relations
Staff allocation	People and jobs	Possible assignments

A *graph* keeps exactly the relationships that matter. Different real situations share the same mathematical structure, so one model and one algorithm serve many fields. We will illustrate everything with a small computer network: a monitoring system must send security logs from a sensor s to an analysis server t through relay machines a and b . Machines become nodes; allowed transfers become arcs. Which allowed route gets a log to the server fastest? This is the same question as finding a fast route between cities.

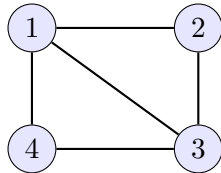
Directed and undirected graphs

Definition 1.6.1 (Directed graph). A *directed graph* (digraph) is a pair $D = (N, A)$ where N is a finite set of *nodes* and $A \subseteq N \times N$ is a set of *arcs*. An arc (i, j) goes *from* i *to* j ; its reverse (j, i) need not exist. We write $n = |N|$ and $m = |A|$. In routing examples, s is the *source* and t the *sink*.



Here $N = \{s, a, b, t\}$, $A = \{(s, a), (s, b), (a, b), (a, t), (b, t)\}$, $n = 4$, $m = 5$.

Definition 1.6.2 (Undirected graph). An *undirected graph* is a pair $G = (V, E)$ where V is a finite set of *vertices* and E is a set of *edges*, each edge being an unordered pair $e = \{i, j\}$ of distinct vertices. Note $\{i, j\} = \{j, i\}$: there is no orientation. (When multiple edges between the same pair are needed, as in Königsberg, E is taken to be a multiset.)



Here $V = \{1, 2, 3, 4\}$ and $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 1\}, \{1, 3\}\}$: candidate physical links between sites, where we care only whether two sites are connected.

What leaves, what enters, what touches

Definition 1.6.3 (Incidence sets). In a digraph $D = (N, A)$, for a node i :

$\delta^+(i) = \{(i, j) \in A\}$ (outgoing arcs), $\delta^-(i) = \{(j, i) \in A\}$ (incoming arcs).

In an undirected graph $G = (V, E)$, for a vertex v : $\delta(v) = \{e \in E : v \in e\}$ is

the set of edges *incident* to v , and $\deg v = |\delta(v)|$ is the *degree* of v . For a set of vertices $S \subseteq V$:

$$\begin{aligned}\delta(S) &= \{\{i, j\} \in E : \text{exactly one of } i, j \text{ is in } S\} \quad (\text{boundary edges}), \\ E(S) &= \{\{i, j\} \in E : i, j \in S\} \quad (\text{internal edges}).\end{aligned}$$

In a digraph, $\delta^+(S)$ is the set of arcs leaving S (tail in S , head outside) and $\delta^-(S)$ the set entering S .

These are sets of *arcs or edges*, not sets of neighbours. In the digraph above, $\delta^+(a) = \{(a, b), (a, t)\}$ and $\delta^-(a) = \{(s, a)\}$. In the undirected graph, $\delta(1) = \{\{1, 2\}, \{1, 3\}, \{1, 4\}\}$, so $\deg 1 = 3$; for $S = \{1, 2\}$, $\delta(S) = \{\{1, 4\}, \{1, 3\}, \{2, 3\}\}$ and $E(S) = \{\{1, 2\}\}$. If no boundary edge of S is selected in a design, the group S is isolated from the rest.

Lemma 1.6.4 (Handshake lemma). *In any undirected graph, $\sum_{v \in V} \deg v = 2|E|$. Consequently the number of odd-degree vertices is even. In any digraph, $\sum_{i \in N} |\delta^+(i)| = \sum_{i \in N} |\delta^-(i)| = |A|$.*

Proof. Count pairs (v, e) with $v \in e$. Summing over vertices counts $\deg v$ for each v ; summing over edges counts 2 for each edge (an edge has two endpoints). Hence $\sum_v \deg v = 2|E|$. The right side is even, so the number of odd terms on the left is even. In a digraph, each arc has exactly one tail and one head, so it is counted once in $\sum_i |\delta^+(i)|$ and once in $\sum_i |\delta^-(i)|$. $\square$

Key insight

Double counting: count the same set of pairs two ways. It is the single most useful proof technique in elementary graph theory, and it reappears in the flow arguments below as “sum the conservation equations.”

Paths, cycles, connectivity, weights

Definition 1.6.5 (Walks, paths, cycles, connectivity). A *path* is a sequence of vertices $v_0, v_1, \dots, v_k$ in which consecutive vertices are joined by an edge (or, in a digraph, by an arc (v_{i-1}, v_i) pointing the way we travel) and no vertex repeats. A *cycle* is such a sequence with $v_k = v_0$, $k \geq 3$, and no other repetition. A graph is *connected* if every pair of vertices is joined by a path.

For instance, in a triangle 1, 2, 3 with an extra vertex 4 attached to 3: 1, 2, 3 is a path; 1, 2, 3, 1 is a cycle; the graph is connected.

Definition 1.6.6 (Weights). A *weight* is a number attached to each arc or edge, e.g. w_{ij} on arc (i, j) . The weight of a path is the sum of the weights of its arcs. Always state the meaning and the units of the weights (time in ms, cost in MAD, capacity in MB/s, ...); the same graph can carry different weights for different questions.

In the network with latencies $w_{sa} = 2$, $w_{at} = 4$, $w_{sb} = 5$, $w_{bt} = 2$ (ms), the path $s \rightarrow a \rightarrow t$ takes $2 + 4 = 6$ ms.

Variables on links and a summation shorthand

The graph and its weights are *data*. The *decisions* live on the links:

- choose whether to use an arc: $x_{ij} \in \{0, 1\}$, e.g. $x_{sa} = 1$ selects (s, a) ;
- choose an amount on an arc: $x_{ij} \geq 0$, e.g. $x_{sa} = 2.5$ sends 2.5 units along (s, a) .

The meaning and domain of x depend on the problem.

Definition 1.6.7 (Sum over a set of links). For a set F of arcs or edges, $x(F) = \sum_{e \in F} x_e$.

For example $x(\delta^+(a)) = x_{ab} + x_{at}$ is the total leaving a ; if $x_{ab} = 1$ and $x_{at} = 2$ the total is 3. For a group S , $x(\delta(S))$ is the total use of boundary edges. First identify the set of links; then add their variables.

Four questions, four graph models

Shortest path	Which route uses the fewest links (or least total weight)?
Maximum flow	How much traffic can reach the sink?
Minimum-cost flow	How cheaply can we send a required amount?
Assignment	Which analyst should handle each incident?

For each we write decisions, objective and constraints, and we prove the basic structural facts.

Shortest path

Definition 1.6.8 (Shortest path as an integer program). Send one unit from s to t using the fewest links. Decision $x_{ij} \in \{0, 1\}$ for each arc ($1 =$ use it). For the example network:

$$\begin{array}{ll}
 \min & x_{sa} + x_{sb} + x_{ab} + x_{at} + x_{bt} \\
 \text{s.t.} & x_{sa} + x_{sb} = 1 && \text{leave the source,} \\
 & x_{ab} + x_{at} - x_{sa} = 0 && \text{at } a : \text{out} = \text{in,} \\
 & x_{bt} - x_{sb} - x_{ab} = 0 && \text{at } b : \text{out} = \text{in,} \\
 & x_{at} + x_{bt} = 1 && \text{reach the sink.}
 \end{array}$$

In general the constraints read $x(\delta^+(i)) - x(\delta^-(i)) = 1$ for $i = s$, $= -1$ for $i = t$, $= 0$ otherwise, and the objective is $\sum_{(i,j)} w_{ij}x_{ij}$ for weights w (all equal to 1 when counting links).

Proposition 1.6.9. For every directed s - t path P , the vector x^P with $x_{ij}^P = 1$ on the arcs of P and 0 elsewhere is feasible for the shortest-path program, with objective value equal to the number of arcs of P (or its weight). Hence the optimal value is at most the length of any s - t path.

Proof. On a path $s = v_0, v_1, \dots, v_k = t$ with no repeated vertex, s has exactly one outgoing path arc and no incoming one, t has one incoming and no outgoing, and every intermediate v_i has exactly one incoming arc (v_{i-1}, v_i) and one outgoing arc

(v_i, v_{i+1}) . So $x^P(\delta^+(i)) - x^P(\delta^-(i))$ equals 1, -1 , 0 as required, and the objective sums the arcs of P . $\square$

In the example, $x_{sa} = x_{at} = 1$, all others 0, is the route $s \rightarrow a \rightarrow t$ with value 2, which beats $s \rightarrow a \rightarrow b \rightarrow t$ (value 3). Since any s - t path has at least 2 arcs (there is no arc (s, t)), the value 2 is optimal, by matching bounds again. All expressions are linear, so this is an integer linear program.

Maximum flow

Definition 1.6.10 (Maximum flow). Each arc (i, j) has a *capacity* $u_{ij} \geq 0$ (e.g. MB/s). A *flow* is a vector x with $0 \leq x_{ij} \leq u_{ij}$ satisfying *conservation* $x(\delta^+(i)) = x(\delta^-(i))$ at every node $i \notin \{s, t\}$: every relay forwards all the traffic it receives. Its *value* is $x(\delta^+(s)) - x(\delta^-(s))$, the net amount leaving the source. The *maximum flow problem* asks for a flow of maximum value:

$$\max x(\delta^+(s)) - x(\delta^-(s)) \quad \text{s.t. conservation at } i \notin \{s, t\}, \quad 0 \leq x_{ij} \leq u_{ij}.$$

Flow may split across several routes; we choose amounts, not a single route. All expressions are linear: this is an LP.

Lemma 1.6.11 (What leaves s arrives at t). *For every flow x , $x(\delta^+(s)) - x(\delta^-(s)) = x(\delta^-(t)) - x(\delta^+(t))$.*

Proof. Sum $x(\delta^+(i)) - x(\delta^-(i))$ over all nodes $i \in N$. Each arc (i, j) contributes $+x_{ij}$ to the term of i and $-x_{ij}$ to the term of j , so the total is 0. Conservation makes every term with $i \notin \{s, t\}$ vanish, leaving $(x(\delta^+(s)) - x(\delta^-(s))) + (x(\delta^-(t)) - x(\delta^+(t))) = 0$, which rearranges to the claim. $\square$

Theorem 1.6.12 (Cuts bound flows). *Let $S \subseteq N$ with $s \in S$ and $t \notin S$ (an s - t cut). For every flow x ,*

$$\text{value}(x) = x(\delta^+(S)) - x(\delta^-(S)) \leq u(\delta^+(S)) := \sum_{(i,j) \in \delta^+(S)} u_{ij}.$$

Hence the maximum flow value is at most the capacity of every cut.

Proof. Sum $x(\delta^+(i)) - x(\delta^-(i))$ over $i \in S$. An arc with both ends in S contributes $+x_{ij}$ (at its tail) and $-x_{ij}$ (at its head), cancelling. An arc leaving S contributes $+x_{ij}$ only; an arc entering S contributes $-x_{ij}$ only. So the sum equals $x(\delta^+(S)) - x(\delta^-(S))$. On the other hand, conservation kills every term except $i = s$, so the sum equals $\text{value}(x)$. Finally $x(\delta^+(S)) \leq u(\delta^+(S))$ since $x_{ij} \leq u_{ij}$, and $x(\delta^-(S)) \geq 0$. $\square$

Key insight

Summing the conservation equations over a set S telescopes: internal arcs cancel and only boundary arcs survive. This is the graph version of “combine the constraints with weights 1 on S , 0 elsewhere,” i.e. weak duality once more.

Example 1.6.13 (Maximum flow in the log network). Capacities (MB/s): $u_{sa} = 4$, $u_{sb} = 3$, $u_{ab} = 2$, $u_{at} = 2$, $u_{bt} = 4$. *Upper bound.* The cut $S = \{s, a, b\}$

has $\delta^+(S) = \{(a, t), (b, t)\}$ with capacity $2 + 4 = 6$, so no flow exceeds 6. A flow of value 6. Send $x_{sa} = 4$, split at a into $x_{at} = 2$, $x_{ab} = 2$; send $x_{sb} = 2$; then $x_{bt} = x_{sb} + x_{ab} = 4$. All capacities respected, conservation holds at a ($4 = 2 + 2$) and at b ($2 + 2 = 4$). Value $= x_{sa} + x_{sb} = 6$. By matching bounds, 6 MB/s is the maximum. (The full max-flow/min-cut theorem, proved later in the course, says a matching cut *always* exists.)

Minimum-cost flow

Definition 1.6.14 (Minimum-cost flow). Each arc has a cost c_{ij} per unit and a capacity u_{ij} . Each node has a balance b_i : $b_i > 0$ is a supply, $b_i < 0$ a demand, $b_i = 0$ a relay. Find the cheapest flow meeting all balances:

$$\begin{aligned} \min \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \text{s.t.} \quad & x(\delta^+(i)) - x(\delta^-(i)) = b_i \quad (i \in N), \\ & 0 \leq x_{ij} \leq u_{ij} \quad ((i,j) \in A). \end{aligned}$$

Maximum flow *chooses* the amount; here the amount is *given*. For a required 4 MB/s from s to t : $b_s = 4$, $b_t = -4$, $b_a = b_b = 0$.

Proposition 1.6.15 (Balances must sum to zero). *If the minimum-cost flow problem is feasible, then $\sum_{i \in N} b_i = 0$.*

Proof. Sum the balance equations over all nodes: the left side is $\sum_i (x(\delta^+(i)) - x(\delta^-(i))) = 0$ (each arc appears once with $+$ and once with $-$, as in Lemma 1.6.11); the right side is $\sum_i b_i$. $\square$

Example 1.6.16 (Minimum-cost flow in the log network). Labels (cost; capacity): (s, a) : 1; 4, (s, b) : 3; 3, (a, b) : 1; 2, (a, t) : 4; 2, (b, t) : 1; 4. Send 4 units from s to t . The three s - t paths cost $s \rightarrow a \rightarrow b \rightarrow t$: 3, $s \rightarrow b \rightarrow t$: 4, $s \rightarrow a \rightarrow t$: 5 per unit. The cheapest path is limited by $u_{ab} = 2$. *Plan*: 2 units on $s \rightarrow a \rightarrow b \rightarrow t$ and 2 units on $s \rightarrow b \rightarrow t$: $x_{sa} = 2$, $x_{ab} = 2$, $x_{sb} = 2$, $x_{bt} = 4$, $x_{at} = 0$. Capacities hold ($x_{bt} = 4 \leq 4$, $x_{sb} = 2 \leq 3$), balances hold. Cost $= 2 \cdot 3 + 2 \cdot 4 = 14$. *Lower bound*. Use the balance equations to eliminate variables. Conservation gives $x_{sa} = x_{ab} + x_{at}$ and $x_{bt} = x_{sb} + x_{ab}$, and the demand at t gives $x_{at} + x_{bt} = 4$, so $x_{sb} = 4 - x_{at} - x_{ab}$. Substituting into the cost,

$$x_{sa} + 3x_{sb} + x_{ab} + 4x_{at} + x_{bt} = 3x_{ab} + 5x_{at} + 4x_{sb} = 16 + x_{at} - x_{ab} \geq 16 + 0 - 2 = 14,$$

using $x_{at} \geq 0$ and the capacity $x_{ab} \leq 2$. Every feasible flow costs at least 14, so the plan is optimal by matching bounds. (Intuitively: the cheap path $s \rightarrow a \rightarrow b \rightarrow t$ at 3 per unit can carry only 2 units because of $u_{ab} = 2$; the rest costs at least 4 per unit.)

Assignment

Definition 1.6.17 (Assignment). A *bipartite graph* has vertex set $L \cup R$ with every edge joining L to R . With cost c_{ij} on edge $\{i, j\}$ (e.g. estimated handling time), the *assignment problem* chooses a set of edges such that every vertex

on both sides belongs to exactly one chosen edge, at minimum total cost:

$$\begin{aligned}
 \min \quad & \sum_{\{i,j\} \in E} c_{ij} x_{ij} \\
 \text{s.t.} \quad & \sum_{j: \{i,j\} \in E} x_{ij} = 1 & (i \in L), \\
 & \sum_{i: \{i,j\} \in E} x_{ij} = 1 & (j \in R), \\
 & x_{ij} \in \{0, 1\} & (\{i, j\} \in E).
 \end{aligned}$$

Example 1.6.18 (Analysts and incidents). Analysts A, B and incidents 1, 2 with handling times $c_{A1} = 2, c_{A2} = 5, c_{B1} = 4, c_{B2} = 3$ hours. Constraints: $x_{A1} + x_{A2} = 1, x_{B1} + x_{B2} = 1$ (each analyst handles one incident); $x_{A1} + x_{B1} = 1, x_{A2} + x_{B2} = 1$ (each incident handled once). There are only two complete assignments: $\{A1, B2\}$ costing $2 + 3 = 5$, and $\{A2, B1\}$ costing $5 + 4 = 9$. Optimal: $A \rightarrow 1, B \rightarrow 2$, total 5 hours.

Proposition 1.6.19 (Assignment requires equal sides). *If the assignment problem is feasible then $|L| = |R|$, and the number of chosen edges is $|L|$.*

Proof. Sum the L -constraints to get $\sum_{i \in L} \sum_j x_{ij} = |L|$, and sum the R -constraints to get $\sum_{j \in R} \sum_i x_{ij} = |R|$. Both sums count every chosen edge exactly once, so they are equal: $|L| = |R|$, and this common value is the number of chosen edges. $\square$

Technique: Writing graph models

- *Data vs. decision.* Nodes, arcs, weights, capacities, balances are data. Variables live on arcs: $x_{ij} \in \{0, 1\}$ to select, $x_{ij} \geq 0$ to send an amount.
- *One constraint per node.* Balance/conservation constraints have the shape $x(\delta^+(i)) - x(\delta^-(i)) = b_i$. Write $b_s = +1$ (or the supply), $b_t = -1$ (or the demand), 0 at relays. Check $\sum_i b_i = 0$.
- *One constraint per arc.* Capacity: $0 \leq x_{ij} \leq u_{ij}$.
- *Objective.* $\sum_{(i,j)} c_{ij} x_{ij}$: count links ($c \equiv 1$), total time, total cost.
- *Bounds from cuts.* For max flow, any cut's capacity is an upper bound; for shortest path, any path's length is an upper bound; for min-cost flow, "cheapest path times demand" is a lower bound.

Classic problems

- *Is there a graph with degrees 3, 3, 3, 3, 3?* No: the degree sum 15 is odd (Lemma 1.6.4).
- *Max flow with capacities $u_{sa} = 4, u_{sb} = 3, u_{ab} = 2, u_{at} = 2, u_{bt} = 4$.* Cut $\{s, a, b\}$ has capacity 6; the flow in Example 1.6.13 has value 6. Answer: 6.
- *Shortest path with unit weights from s to t when no arc (s, t) exists.* At least 2 arcs; any 2-arc path is optimal.

- *Assignment* 2×2 . Compare the two diagonals of the cost matrix. Optimal is the cheaper diagonal.

Gotcha

- $\delta^+(i)$ is a set of *arcs*, not of neighbours. Count arcs, then sum their variables.
- In a directed path every arc must point the way you travel. $s \rightarrow b$, then “ $b \rightarrow a$ ” is illegal if only (a, b) exists.
- Conservation is *not* imposed at s and t in max flow; imposing it would force value 0.
- In min-cost flow the objective is $\sum c_{ij}x_{ij}$ with cost per unit; do not multiply by capacities.

Summary: Graph models

- Digraph $D = (N, A)$ with ordered arcs; undirected graph $G = (V, E)$ with unordered edges. Incidence sets $\delta^+(i), \delta^-(i), \delta(v), \delta(S), E(S)$ are sets of links; $x(F) = \sum_{e \in F} x_e$.
- Handshake: $\sum \deg v = 2|E|$; summing conservation over a node set telescopes to the boundary.
- Four models: shortest path (ILP, select arcs), maximum flow (LP, choose amounts, cuts give upper bounds), minimum-cost flow (LP, given balances summing to 0), assignment (binary, exactly one on each side).
- Graphs and equations describe the same problem; the graph guides the equations.

1.7 Convexity

Motivation

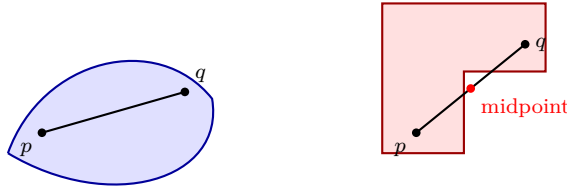
Why are linear programs easy while integer programs are hard? The deepest answer is *convexity*. Two properties of LPs are responsible: any mixture of two feasible plans is feasible, and a linear objective never rewards “hiding” a good solution behind a worse one nearby. Together they imply that a solution which cannot be improved by small changes cannot be improved at all. This section makes that precise.

Convex: if two plans are allowed, every blend of them is allowed. Convex objective: blending never costs more than the blended cost.

Definition 1.7.1 (Convex set). A set $X \subseteq \mathbb{R}^n$ is *convex* if for all $p, q \in X$ and all $\lambda \in [0, 1]$,

$$\lambda p + (1 - \lambda)q \in X.$$

Geometrically: the whole segment between any two points of X stays inside X .



convex: the whole segment stays inside nonconvex: even the midpoint is outside

Proposition 1.7.2 (Feasible sets of linear programs are convex). *Any set of the form $X = \{x \in \mathbb{R}^n : Ax \leq b\}$ (with equality constraints and sign constraints allowed, since $a^\top x = \beta$ is the pair $a^\top x \leq \beta, -a^\top x \leq -\beta$, and $x_j \geq 0$ is $-x_j \leq 0$) is convex.*

Proof. Let $p, q \in X$ and $\lambda \in [0, 1]$. Then $A(\lambda p + (1 - \lambda)q) = \lambda Ap + (1 - \lambda)Aq \leq \lambda b + (1 - \lambda)b = b$, where the inequality uses $Ap \leq b, Aq \leq b$ and $\lambda, 1 - \lambda \geq 0$. So $\lambda p + (1 - \lambda)q \in X$. $\square$

In Example 1.3.8, half of each of two feasible production plans gives their average, again feasible.

Definition 1.7.3 (Convex function). Let X be convex. A function $f : X \rightarrow \mathbb{R}$ is *convex* if for all $p, q \in X$ and $\lambda \in [0, 1]$,

$$f(\lambda p + (1 - \lambda)q) \leq \lambda f(p) + (1 - \lambda)f(q).$$

Geometrically: between any two points, the graph of f lies on or below the chord joining them.

Example 1.7.4 ($f(x) = x^2$). Choose $x = 0$ or $x = 2$: costs 0 and 4, average cost 2. The cost at the average choice is $f(1) = 1 \leq 2$: the curve lies below the joining line. In general, for $\lambda \in [0, 1]$,

$$\lambda p^2 + (1 - \lambda)q^2 - (\lambda p + (1 - \lambda)q)^2 = \lambda(1 - \lambda)(p - q)^2 \geq 0,$$

(expand and simplify) so x^2 is convex on $\mathbb{R}$.

Proposition 1.7.5 (Linear functions are convex). $f(x) = c^\top x + d$ is convex (and so is $-f$; such functions are called affine).

Proof. $f(\lambda p + (1 - \lambda)q) = \lambda c^\top p + (1 - \lambda)c^\top q + d = \lambda f(p) + (1 - \lambda)f(q)$: equality holds, so in particular $\leq$ holds. The same computation applies to $-f$. $\square$

Definition 1.7.6 (Local and global minima). Let $f : X \rightarrow \mathbb{R}$. A point $x^* \in X$ is a *global minimum* if $f(x^*) \leq f(x)$ for all $x \in X$. It is a *local minimum* if there is $\varepsilon > 0$ such that $f(x^*) \leq f(x)$ for all $x \in X$ with $\|x - x^*\| < \varepsilon$: no better feasible choice nearby.

Theorem 1.7.7 (In convex minimisation, local is global). *Let X be a convex set and $f : X \rightarrow \mathbb{R}$ a convex function. Every local minimum of f over X is a global minimum.*

Proof. Step 1 (set-up). Let x^* be a local minimum with radius ε , and suppose for contradiction that some $y \in X$ has $f(y) < f(x^*)$.

Step 2 (walk toward y). For $\lambda \in (0, 1]$ put $x_\lambda = x^* + \lambda(y - x^*) = (1 - \lambda)x^* + \lambda y$. By convexity of X , $x_\lambda \in X$. By convexity of f ,

$$f(x_\lambda) \leq (1 - \lambda)f(x^*) + \lambda f(y) < (1 - \lambda)f(x^*) + \lambda f(x^*) = f(x^*),$$

where the strict inequality uses $f(y) < f(x^*)$ and $\lambda > 0$.

Step 3 (contradiction). $\|x_\lambda - x^*\| = \lambda\|y - x^*\|$, which is smaller than ε for λ small enough. So for such λ , x_λ is a feasible point within distance ε of x^* with a strictly smaller objective, contradicting local minimality. Hence no such y exists and x^* is a global minimum. $\square$

Key insight

Convexity turns a *far-away* better point y into *nearby* better points: the segment from x^* to y stays feasible, and the objective along it stays below the chord, so it decreases immediately. A convex landscape has no “valleys” in which to get trapped.

Corollary 1.7.8 (Linear programming is convex minimisation). *For an LP, the feasible set is convex (Proposition 1.7.2) and the objective is convex (Proposition 1.7.5; for maximisation, minimise $-c^\top x$, also convex). Hence every local optimum of an LP is a global optimum.*

Proof. Combine the two propositions with Theorem 1.7.7. $\square$

Proposition 1.7.9 (The set of optimal solutions of an LP is convex). *Let X be convex, f linear, and $X^* = \{x \in X : f(x) = z^*\}$ the set of optimal solutions of $\min_X f$. Then X^* is convex.*

Proof. For $p, q \in X^*$ and $\lambda \in [0, 1]$, $\lambda p + (1 - \lambda)q \in X$ by convexity of X , and $f(\lambda p + (1 - \lambda)q) = \lambda z^* + (1 - \lambda)z^* = z^*$ by linearity. So the point is optimal. $\square$

Interpretation & discussion

Two consequences shape everything that follows. First, an algorithm that only makes *local improvements* (like the simplex method) can be trusted to find a *global* optimum of an LP. Second, if an LP has two optimal solutions it has infinitely many (the whole segment between them). Integer programs break both: the feasible set $\{0, 1\}^n$ is not convex (the midpoint of two integer points is usually fractional), so local search can get stuck and the theory above gives no guarantee.

Technique: Checking convexity

- *Sets:* intersections of convex sets are convex; half-spaces $\{a^\top x \leq \beta\}$ and hyperplanes are convex; so any system of linear (in)equalities defines a convex set. A set described with “or” ($x \leq -1$ or $x \geq 1$) is usually not convex: test the midpoint.
- *Functions:* linear/affine functions, x^2 , $|x|$, e^x are convex; sums of convex functions with nonnegative weights are convex; the maximum of convex functions is convex. A twice-differentiable $f : \mathbb{R} \rightarrow \mathbb{R}$ is convex iff $f'' \geq 0$.

- *Quick disproof:* find two points where the midpoint fails. For $f(x) = -x^2$: $f(0) = 0 > \frac{f(-1)+f(1)}{2} = -1$, not convex.
- *Maximising a convex function* is *not* convex minimisation: $\max xy$ in the garden example needed a separate argument.

Classic problems

- *Is $\{(x, y) : xy \geq 1, x, y > 0\}$ convex?* Yes: it is the region on or above the graph of $g(x) = 1/x$ on $x > 0$, and g is convex there ($g''(x) = 2/x^3 > 0$). The region above the graph of a convex function is convex: if $y_1 \geq g(x_1)$ and $y_2 \geq g(x_2)$ then $\lambda y_1 + (1-\lambda)y_2 \geq \lambda g(x_1) + (1-\lambda)g(x_2) \geq g(\lambda x_1 + (1-\lambda)x_2)$.
- *Is $\{x \in \mathbb{R} : |x| \geq 1\}$ convex?* No: -1 and 1 are in the set, their midpoint 0 is not.
- *An LP has optimal solutions $(0, 4)$ and $(2, 2)$. Give a third.* $(1, 3)$, by Proposition 1.7.9.

Gotcha

- “Convex” is a property of the *pair* (set, objective) for minimisation. A convex objective over a nonconvex set (integer points) or a concave objective to be minimised both lose the local-is-global guarantee.
- x^2 is convex, but $\max x^2$ over $[-1, 2]$ is attained at an endpoint, not where the derivative vanishes; the theorem is about *minimising* convex functions.

Summary: Convexity

- Convex set: segments stay inside. Convex function: graph below chords.
- Feasible sets of linear systems are convex; linear objectives are convex.
- In convex minimisation every local minimum is global (Theorem 1.7.7); LPs are a special case, and their optimal sets are convex.

Notes on the original slides

Every number in the Lecture 1 slides was recomputed while writing this chapter, and all of them check out (the transport bounds 96,000 and 92,000, the certificate $30x + 20y = 10(x+y) + 10(2x+y)$, the knapsack and project tables, the incidence sets, the flow constraints, the assignment costs). A few statements are correct but easy to misread, and are worth a warning.

- **“A model can have different outcomes” (slide 26).** The three outcomes listed (an optimum, infeasible, unbounded) are exhaustive for *linear* programs (Chapter 2) but not for general mathematical programs: $\min x$ over $x > 0$ is feasible and bounded yet has no optimum. See the remark after Definition 1.2.4.
- **“Path: a sequence of linked vertices” (slide 47).** As stated this also covers walks that revisit vertices. The standard convention, used in

this chapter, is that a path has no repeated vertex; the slides' examples all satisfy it.

- **Shortest-path constraints (slide 53).** The sink equation $x_{at} + x_{bt} = 1$ is implied by the other three (add them). Keeping it is harmless and mirrors the general form $x(\delta^+(i)) - x(\delta^-(i)) = b_i$ at every node, but do not be surprised if a solver reports a redundant row.
- **Lower bound by “cheapest route” (slide 35).** The slide states $L = 92,000$ as if it were obvious; strictly, one must also argue that 92,000 is the *optimum* of the relaxed problem, not just the cost of one relaxed plan. The one-line justification is given in Section 1.5.
- **Knapsack relaxation (slide 29).** Here the LP relaxation happens to give the integer optimum 15; this is a coincidence of the data, not a general fact.
- **Maximum flow objective (slide 55).** $\max x_{sa} + x_{sb}$ is the net outflow of s only because no arc enters s ; in general the objective is $x(\delta^+(s)) - x(\delta^-(s))$.

Chapter takeaways

Summary: From a decision to a model

- **Build the model.** Variables: what can we choose (continuous, integer, binary)? Constraints: what must we respect? Objective: what do we minimise or maximise?
- **Judge the solution.** Feasible: satisfies every constraint. Optimal: best objective value among feasible choices. Bounds: for minimisation, lower bound $\leq$ optimal cost $\leq$ feasible cost; matching bounds prove optimality.
- **Choose a useful representation.** LPs have continuous variables and linear expressions, and any LP can be put in standard form. Convexity makes local optima global. Binary variables select, assign or activate. Graphs (vertices, links, weights with stated units) organise shortest-path, flow and assignment models; the graph and the equations describe the same problem.
- **Check the model.** Does every mathematically allowed choice make sense in reality? Start simple, state assumptions, and check the result against reality.
- **Next.** Solving linear programs (Chapter 2); graph algorithms later.